LOGO!-Änderungen – Kostal-Fix, App-Steuerung, VM-Adressen

Aufbauend auf dem bestehenden Programm `Meyer_PV.lsc` (LOGO! 8.3, 192.168.178.60). Kostal-Anbindung und Display bleiben erhalten. Stand: 24.10.2026.

1. Kostal-Werte richtig lesen (der eigentliche Fehler)

Die Kostal-Leistungswerte sind **32-Bit-Gleitkomma (REAL)** und belegen je **2 Register**. Dein Programm liest aber **Länge 1** (16 Bit), teils am **ungeraden** Register – und die Float-Konverter stehen auf überlappenden VM-Adressen (`VM=2` und `VM=4` überschneiden sich). Ergebnis: nur halbe/vermischte Zahlen → alles Weitere ist Unsinn.

1a. Modbus-Lesebefehle (Verbindung 2, Kostal) korrigieren

Je Wert **Länge 2** ab dem **geraden** Basisregister in ein **Doppelwort** (VD), 4-Byte-Abstand, nicht überlappend:

Wert	bisher (falsch)	neu: entf. Register	Länge	lokal (VD)	Float-Konverter
PV-Erzeugung	HR1067, L1 → VW2	**HR1066**	2	`VD2`	B003: Type=Float, **VM=2**
Hausverbrauch gesamt	HR173, L1 → VW4	**HR172**	2	`VD6`	B004: Float, **VM=6**
Batterie-SOC (%)	HR211, L1 → VW8	**HR210**	2	`VD10`	B005: Float, **VM=10**
Hausverbrauch Batterie	HR107, L1 → VW12	**HR106**	2	`VD14`	B007: Float, **VM=14**
Batterie-SOC (Alternative)	–	**514**	1	`VW…`	U16 (Int), kein Float
Batterie-Leistung	HR583, L1 → VW34	**582**	**1**	`VW26`	**S16 (Int), kein Float!**
Netzleistung (Register 252)	–	**252**	2	`VD28`	Float-Konverter, VM=28 → siehe unten

Belegt bestätigt aus der KOSTAL-Protokollbeschreibung (Stand 2026): PV-Erzeugung = „Total DC power" Reg. **1066** (Float); SOC = Reg. **210** (Float) *oder* Reg. **514** (U16); Hausverbrauch aus Batterie = Reg. **106** (Float); Batterie-Leistung = Reg. **582** (**S16, 1 Register** – als Int lesen); Netzleistung am Netzübergabepunkt = Reg. **252** (Float). „Hausverbrauch gesamt" (Reg. 172) bitte in der Doku gegenprüfen.

Wort-/Byte-Reihenfolge (häufige Fehlerquelle): KOSTAL liefert Floats standardmäßig **little-endian (CDAB)** (einstellbar in Register 0x05). Den **Float-Konverter in der LOGO! entsprechend einstellen**, sonst kommt die Zahl verdreht an. Datentyp je Wert beachten: **Float = Länge 2**, **S16/U16 = Länge 1 (als Int)**. Nach der Korrektur stimmen auch die Werte im LOGO!-Display (Meldetext B002).

Empfohlen für „nur Überschuss": Statt `PV − Hausverbrauch` (Block B001) die **Netzleistung am Netzübergabepunkt direkt lesen: Register 252** („Total active power (powermeter)", Float, 2 Register, W). **Vorzeichen: + = Netzbezug, − = Einspeisung.** Überschuss = **− (Register 252)** – in der LOGO! am einfachsten über einen **Analogverstärker mit Verstärkung −1**. So heizt die Anlage wirklich nur mit echter Netz-Einspeisung (nach Batterie), nicht aus dem Akku.

2. App-Steuerung einbauen (Betriebsart + Hand)

Dein Programm hat bisher keine App-Steuerung. Fünf neue Merker anlegen (die die App über die VM schreibt):

Merker	Bedeutung	von der App
M10	Automatik ein	`DBX…` schreibt 1/0
M11	Hand-Modus ein	1/0
M12 / M13 / M14	Hand Stufe 1 / 2 / 3	1/0

Ausgangslogik ändern

Bei dir treiben die Zeitglieder direkt die Ausgänge: `B018→Q1`, `B015→Q2`, `B017→Q3`. Diese Verbindung auftrennen und je Stufe dazwischenschalten:

```
Q1 = (M10 ∧ ¬M11 ∧ B018) ∨ (M11 ∧ M12)
Q2 = (M10 ∧ ¬M11 ∧ B015) ∨ (M11 ∧ M13)
Q3 = (M10 ∧ ¬M11 ∧ B017) ∨ (M11 ∧ M14)
```

Pro Stufe also: ein **UND** (Automatik-Zweig, mit ¬M11), ein **UND** (Hand-Zweig) und ein **ODER** auf den Ausgang. `¬M11` einmal über einen **NICHT**-Block bilden und dreifach verwenden. So hat Hand Vorrang, und ohne Automatik/Hand sind alle Stufen aus.

Status für die App: Merker **M20 = M10 ∧ ¬M11** („Automatik aktiv"); „Hand aktiv" ist M11 selbst.

Tipp zu den Zeiten: Deine Zeitglieder stehen auf 10–13 min Ein-/Ausschaltverzögerung. Das ist sehr träge. Für zügigeres Schalten auf z. B. **60–120 s** Einschalt- und **30–60 s** Ausschaltverzögerung stellen (B018/B015/B017). Die Zeiten sind über deine Parameter-VM-Zuordnung (VM 60–110) ohnehin schon von außen änderbar.

3. VM-Adressen für die App

Gut: Alle Werte (Analog-Merker `AM`, Ausgänge `Q`, Merker `M`) liegen in der LOGO! **ohne Zusatzblöcke** bereits in der VM und sind damit über S7 als `DB1` lesbar. Du musst nur die richtigen VM-Adressen in die App eintragen.

So abzulesen: in LOGO!Soft Comfort die **VM-/Modbus-Adresstabelle** öffnen – dort steht für jedes `AM`, `Q`, `M` die VM-Adresse. Diese Adresse in der App beim jeweiligen Tag als `DB1`-Offset eintragen (die App-Adressen sind frei editierbar).

App-Wert	LOGO!-Quelle	VM-Adresse (typisch, bestätigen!)	App-Tag
PV-Erzeugung	B003 → AM4	`VW958`	PV, Int

Hausverbrauch	B004 → AM5	`VW960`	Hausverbrauch, Int
Batterie-SOC	B005 → AM6	`VW962`	SOC, Int
Überschuss	B001 → AM3	`VW956`	Ueberschuss, Int
Batterie-Leistung	→ AM22	`VW994`	BatterieLeistung, Int
Stufe 1/2/3 ein	Q1/Q2/Q3	`V942.0 / .1 / .2`	Stufe1/2/3, Bool
Automatik / Hand (schreiben)	M10 / M11	`V948… (M10/M11)`	Automatik/Hand, Bool
Hand Stufe 1/2/3 (schreiben)	M12 / M13 / M14	`V948…`	HandStufe1/2/3, Bool
Auto aktiv	M20	`V948…`	AutoAktiv, Bool
Ein-Schwelle	B010.On	`VW50` (schon gemappt)	EinSchwelle, Int

Die VW-Adressen für AM/Q/M sind die typische LOGO!-8-Werkseinstellung; die **tatsächlichen** Werte stehen in deiner VM-Tabelle. Nenne sie mir, dann trage ich sie exakt in die `projekt.json` ein – oder du passt die Tags in der App direkt an (Reiter „Tags").

4. Display der LOGO!

Der Meldetext **B002** zeigt bereits Überschuss, PV, Hausverbrauch, Batterie-SOC, Batterie-Leistung und die Stufen. Nach dem Modbus-Fix stimmen diese Zahlen. Sinnvoll ist, oben **Überschuss (W)** und die **drei Stufen** gut sichtbar zu halten; eine zweite Textseite kann PV / Hausverbrauch / SOC zeigen.

5. Reihenfolge der Umsetzung

1. **Modbus-Fix** (Abschnitt 1) – zuerst, denn ohne richtige Werte ist alles andere sinnlos. In der App bzw. am Display die Werte gegenprüfen (PV, Hausverbrauch, Überschuss plausibel?).
2. **Zeiten** auf 60–120 s / 30–60 s stellen und einen Automatik-Test machen (künstlich Last erzeugen → Stufen fallen ab).
3. **App-Steuerung** (Abschnitt 2) einbauen und mit der App testen (Hand-Modus, Stufen einzeln).
4. **VM-Adressen** (Abschnitt 3) in der App eintragen – fertiges Bedienbild `projekt.json` ist schon da.

Das `.lsc`-Format ist verschlüsselt und kann nicht von außen erzeugt werden; dies ist die nachbaubare Änderungsanleitung zum bestehenden Programm. Begleitunterlage zum SPS-Monitor-Projekt.